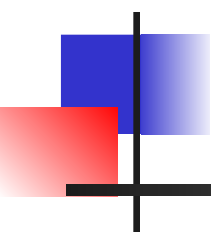




CBCC – Curso de Bacharelado em Ciência da Computação
CBSI – Curso de Bacharelado em Sistemas de Informação



UML – Diagrama de Classes

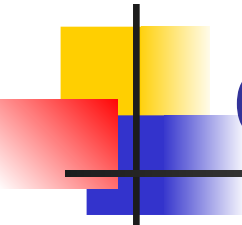
Prof. Dr. Sandro Ronaldo Bezerra Oliveira
srbo@ufpa.br

Engenharia de Software

Faculdade de Computação

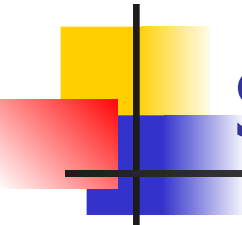
Instituto de Ciências e Exatas e Naturais

Universidade Federal de Pará



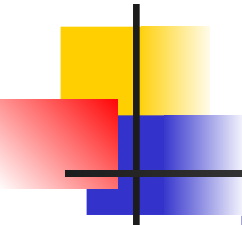
Objetivos

- Notação Básica
- Dicas Práticas
- Exercício



Sobre Classes

- Classe é o elemento mais importante de qualquer sistema orientado a objetos
 - Ela que permite definir a estrutura interna do sistema
- Uma classe é uma descrição de um conjunto de objetos com os mesmos atributos, relacionamentos, operações e semântica.
- Classes são usadas para capturar o vocabulário de um sistema
- Classes podem ser abstrações do domínio do problema, como "Cliente", "Banco", "Conta"
- Classes podem também ser usadas a nível de implementação (listas, filas, eventos).



Nomes

- Toda classe deve ter um nome que a distinga das outras classes
- Um nome pode ser simples (apenas o nome), ou pode ser precedido pelo nome do pacote em que a classe está contida.

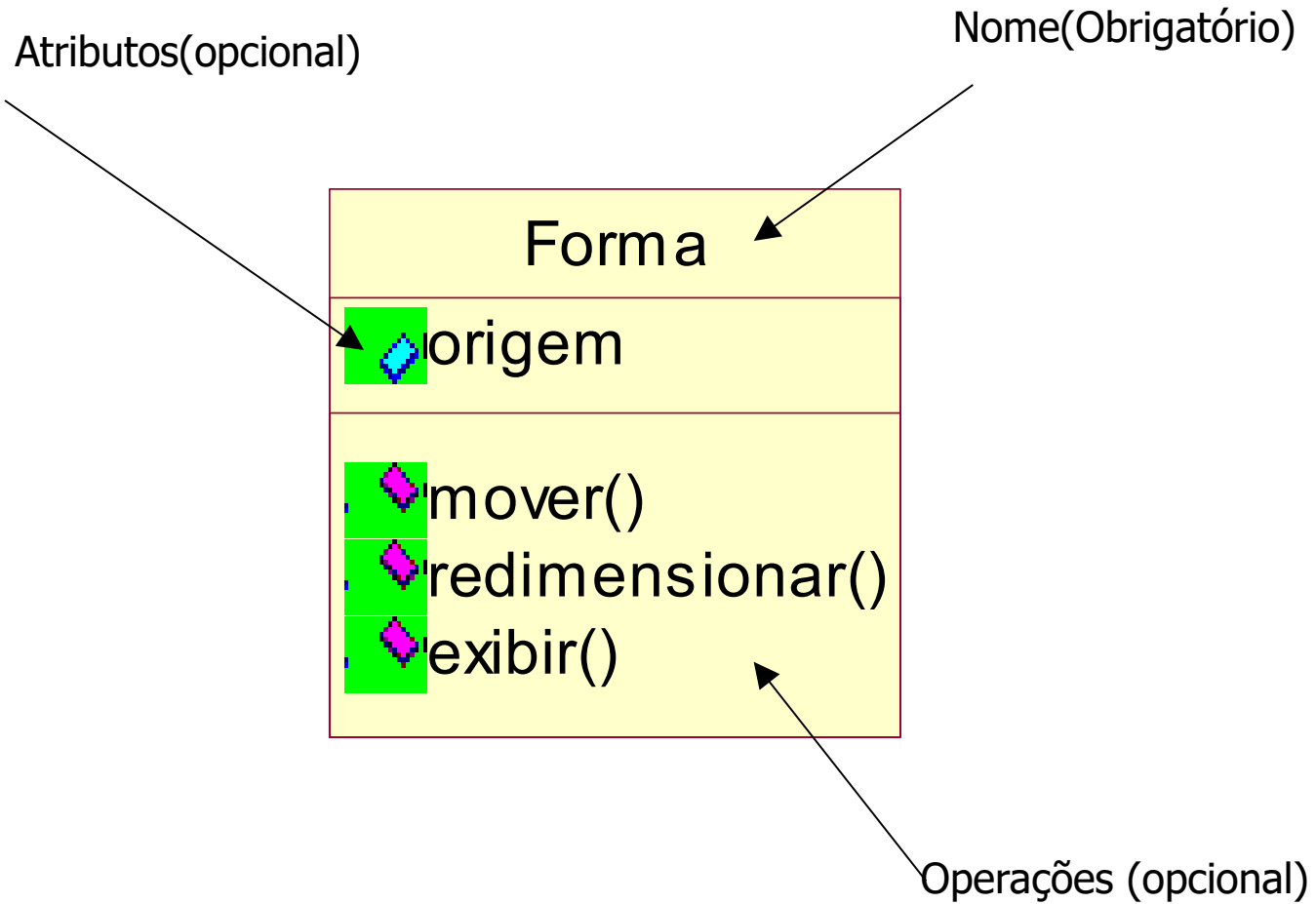
Conta

Banco

Cliente

Exceções::ClienteNãoCadastrado

Notação Básica



Variações

Indivíduo

Indivíduo

 Código : long
 Sexo : char
 Nome : char

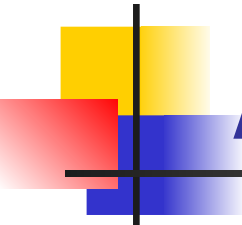
Indivíduo

 incluir()
 atualizar()

Indivíduo

 Código : long
 Sexo : char
 Nome : char

 incluir()
 atualizar()

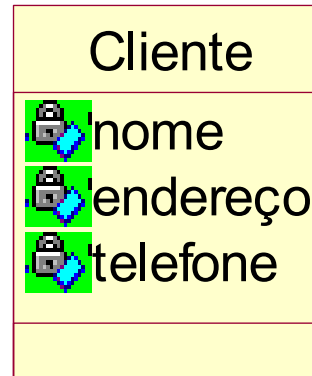


Atributos

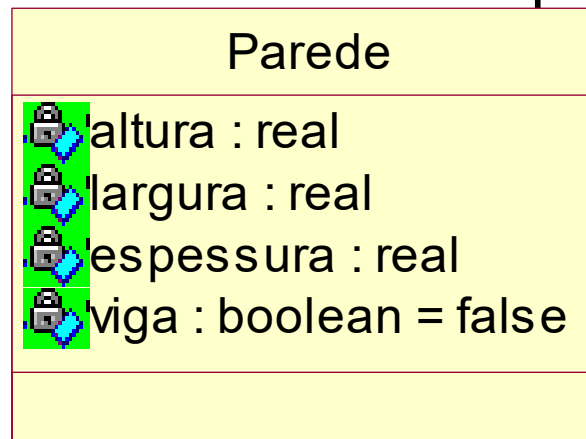
- Um atributo representa alguma propriedade do que está sendo modelado, que é compartilhada por todos os objetos da classe
- Os atributos descrevem os dados contidos nas instâncias de uma classe
- Em um dado momento, um objeto de uma classe conterá valores para todos os atributos descritos na sua classe

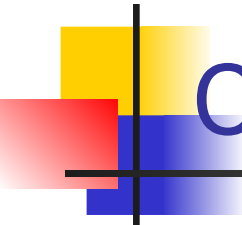
Atributos (Notação)

- Atributos podem ser identificados apenas com nomes



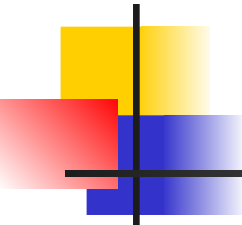
- Atributos podem ter seus tipos (ou classes) especificados e terem valores padrão definidos





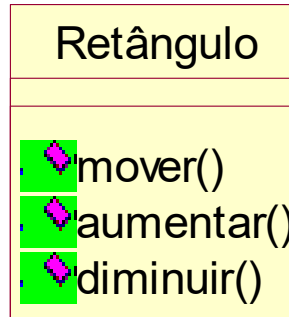
Operações

- Uma operação é uma abstração de alguma coisa que se pode fazer com um objeto e que é compartilhada por todos os objetos da classe
- Uma classe pode ter qualquer número de operações, inclusive nenhuma

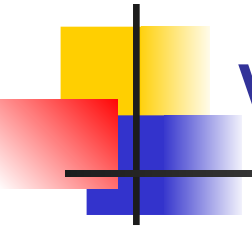


Operações (Notação)

- Como para os atributos, você pode especificar uma operação apenas com seu nome



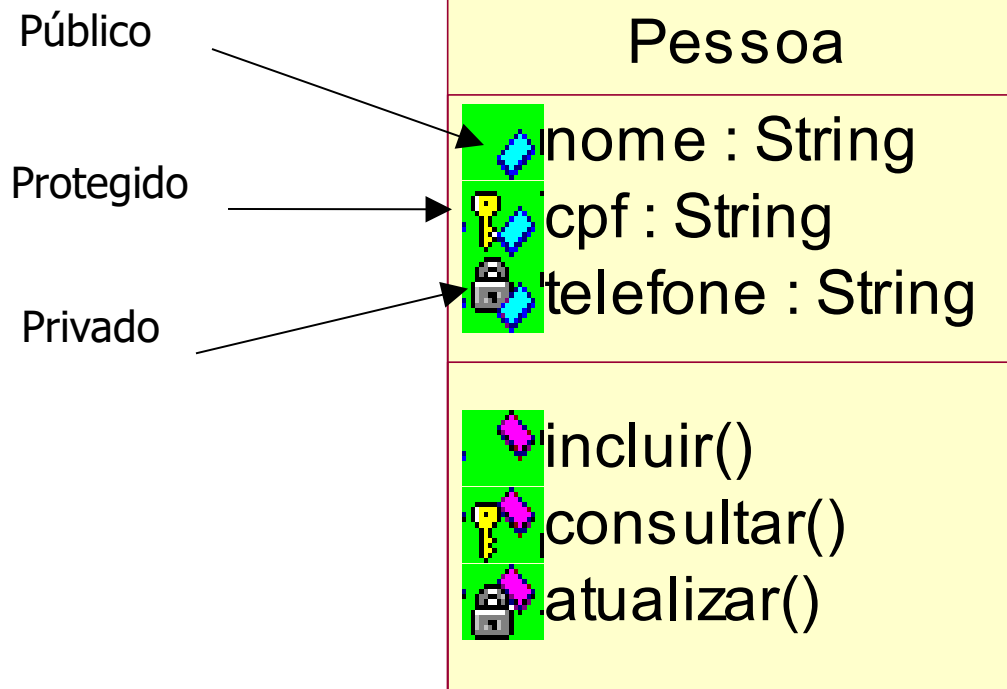
- Você pode também especificar a assinatura da operação: seus parâmetros, o tipo desses parâmetros e o tipo de retorno

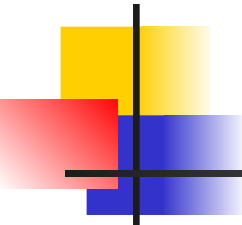


Visibilidade

- Você pode usar marcações de acesso para especificar o tipo de acesso permitido aos atributos e operações
- Classificador: classes, interfaces, componentes, nós, use cases, subsistemas
- Público: todos os classificadores podem usar
- Protegido: qualquer descendente do classificador poderá usar
- Privado: somente o próprio classificador poderá usar

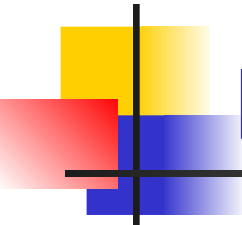
Visibilidade do Rose





Sobre Relacionamentos

- Poucas classes vivem sozinhas
- A maioria das classes colabora com outras classes de várias maneiras
- Tipos de relacionamentos especialmente importantes na modelagem orientada a objetos
 - Dependências
 - Generalizações
 - Associações
 - Agregação
 - Composição
 - Realização



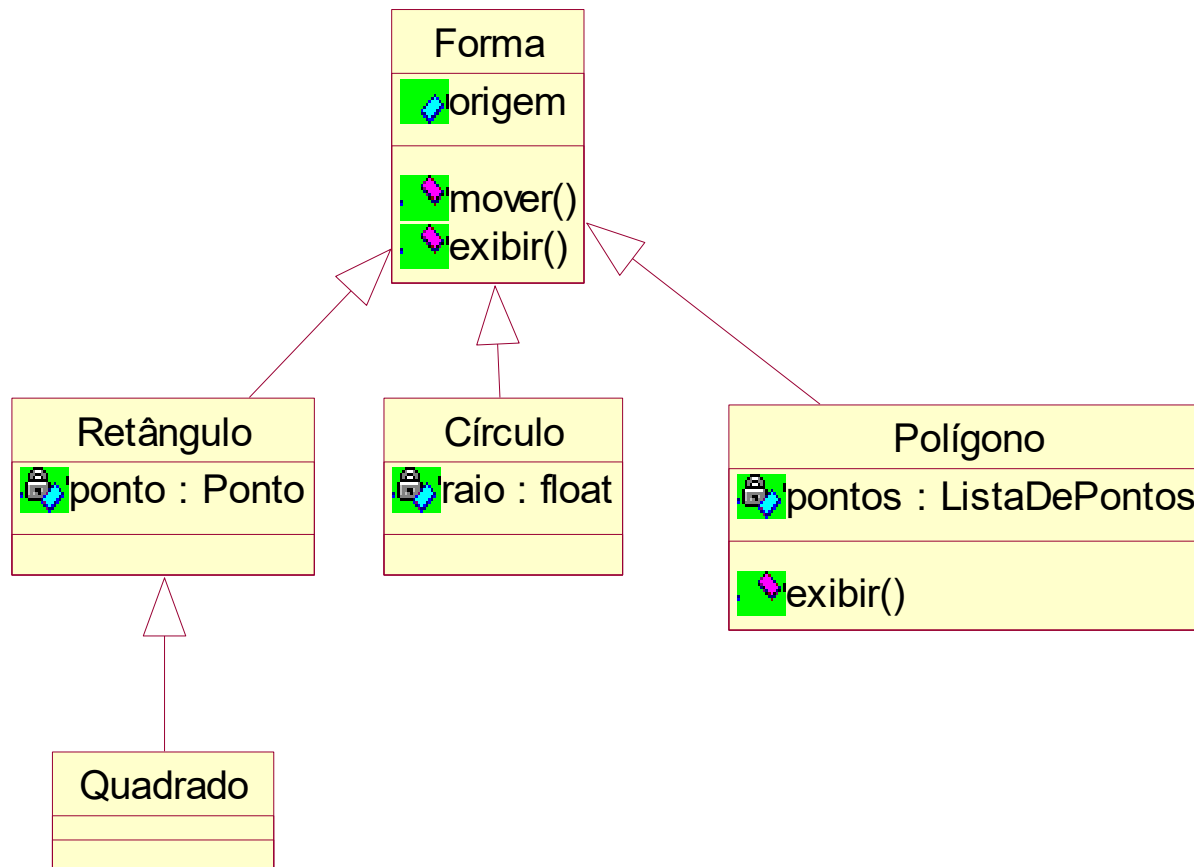
Dependência

- Dependências são relações de uso
- Uma dependência indica que mudanças em um elemento (o "servidor") podem alterar outro elemento (o "cliente")
- Uma dependência entre classes indica que os objetos de uma classe usam serviços dos objetos de outra classe



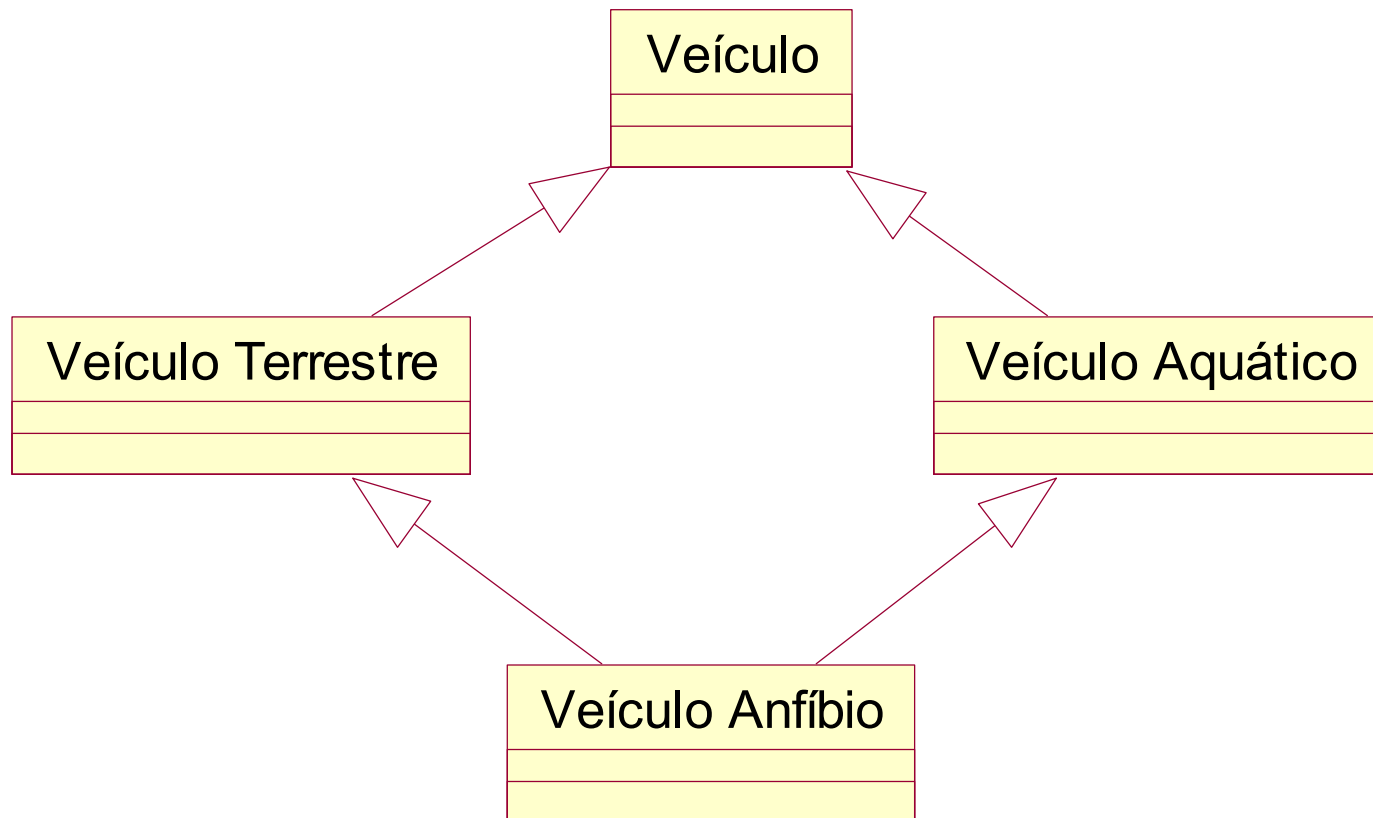
Generalização

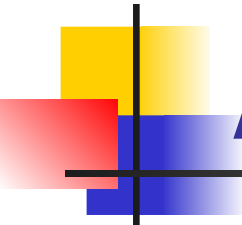
- Uma generalização (também conhecida como herança) é um relacionamento entre um elemento mais geral (chamado de superclasse ou pai) e um mais específico (chamado de subclasse ou filho)



Herança Múltipla

- Ocorrem múltiplas superclasses para uma mesma subclasse

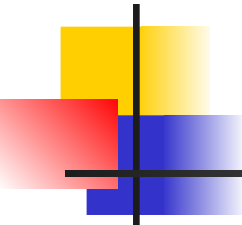




Associação

- A associação é um relacionamento estrutural que especifica que objetos de um elemento estão conectados a objetos de outro elemento

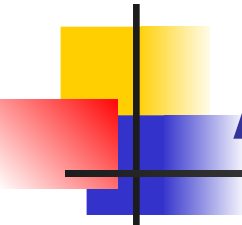




Multiplicidade

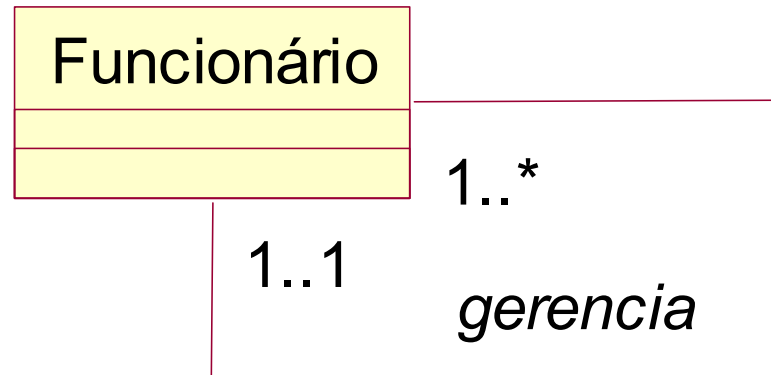
- É a cardinalidade de uma associação

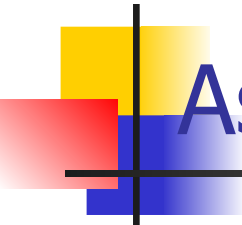
1..1	Classe	Exatamente 1
0..0	Classe	Nenhum (zero ou zero)
0..1	Classe	Opcional (zero ou um)
0..*	Classe	Nenhum ou Muitos (zero ou mais)
1..*	Classe	Um ou Muitos (um ou mais)
*	Classe	Muitos



Associação Unária

- Quando há um relacionamento de uma classe para consigo própria

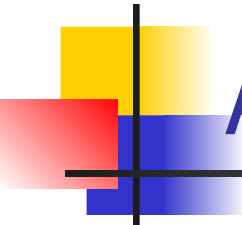




Associação Binária

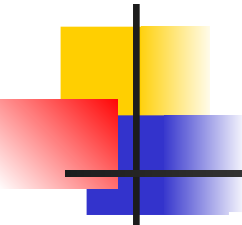
- Quando há duas classes envolvidas na associação de forma direta de uma para a outra



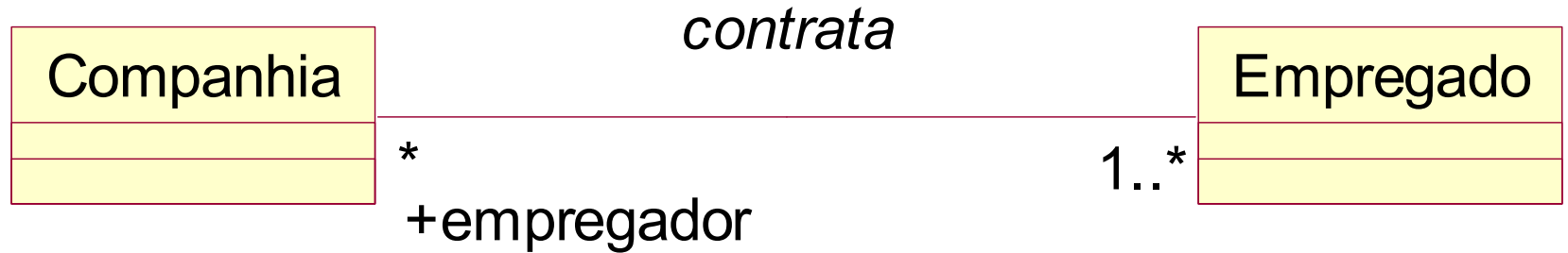


Associação e papéis

- Papéis: um dos lados da associação
- Uma associação binária possui duas “terminações”. Cada uma delas pode possuir um “nome de papel”. Uma associação n-ária pode possuir um “papel” para cada terminação.
- Nomes de papéis são necessários para associação entre dois objetos da mesma classe

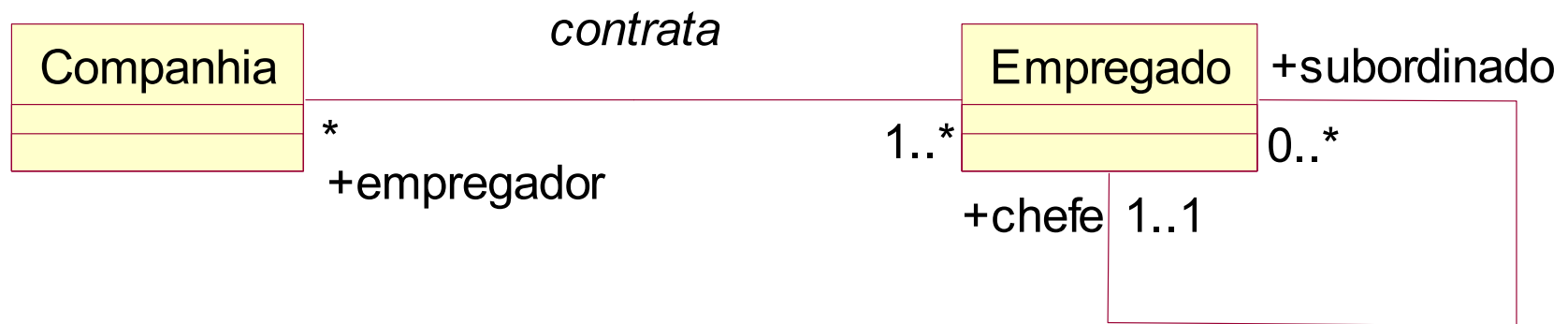


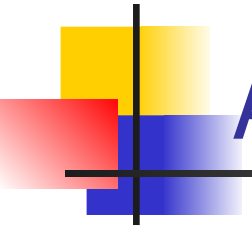
Associação e Papéis



Associação e papéis

- Nome de papéis são úteis para distinguir entre associações entre o mesmo par de classes

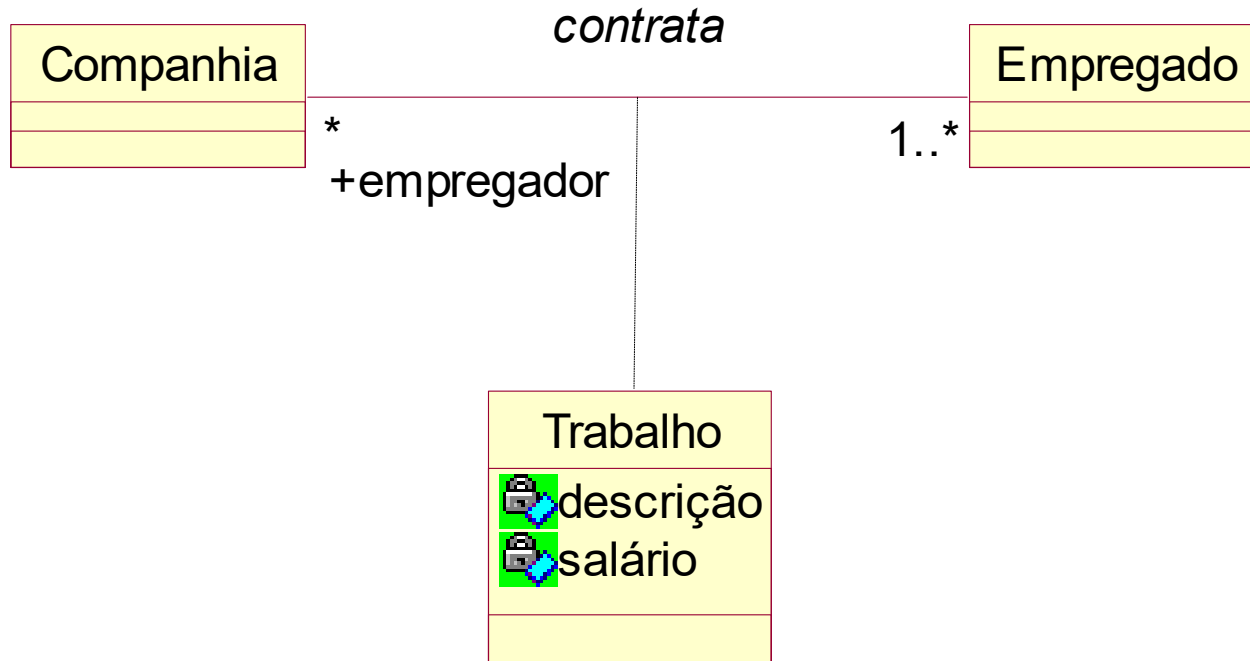




Associação com Atributos

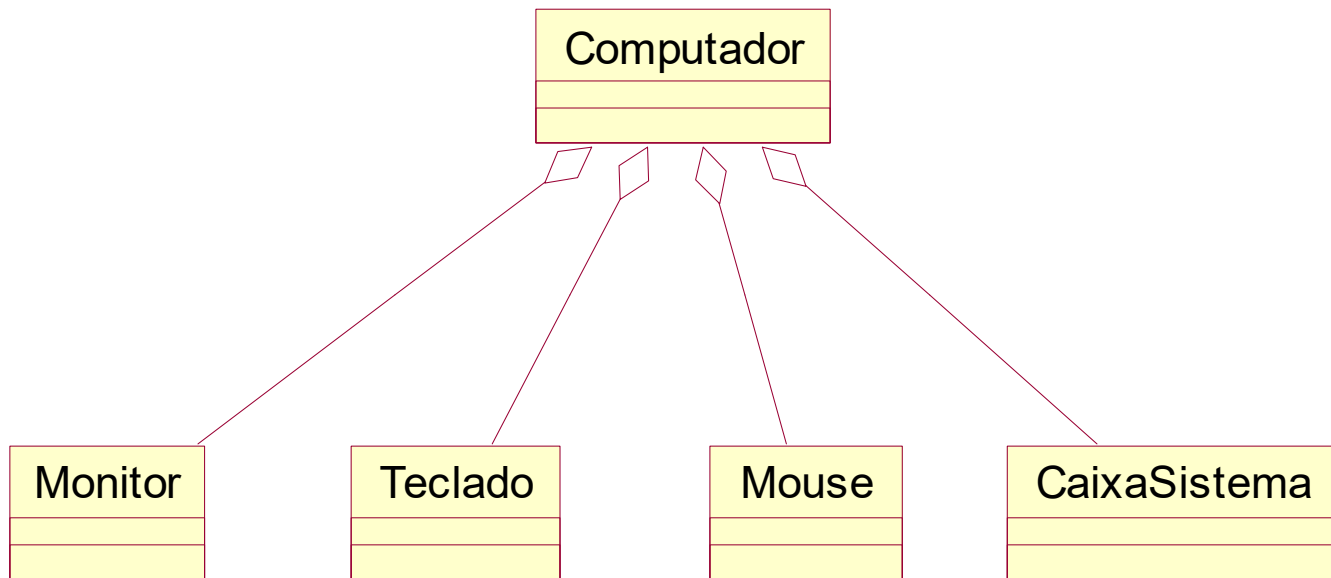
- Modela as propriedades associadas com uma associação
- Para indicar os atributos de uma associação, usamos uma linha tracejada para unir a associação às suas propriedades
- As propriedades devem ser representadas por uma classe

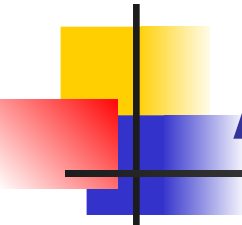
Associação com Atributo



Agregação

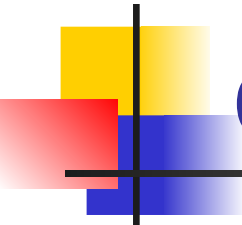
- Uma forma especial de associação entre o todo e suas partes, no qual o todo é composto de partes





Agregação

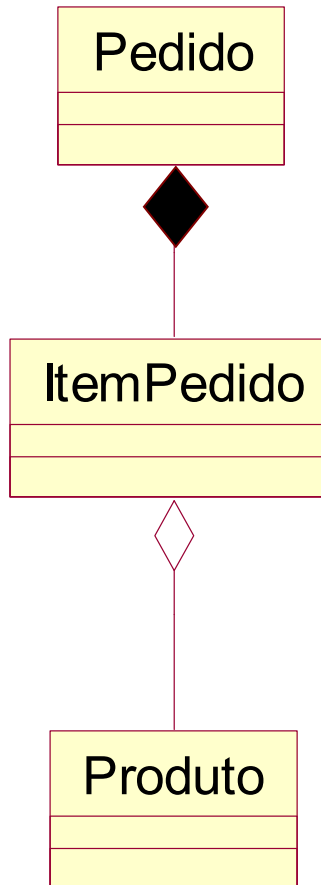
- Distingue um “Todo” de uma “Parte”
- Não muda o significado de navegação entre o “Todo” e as “Partes”
- Não impede que a vida das “Partes” esteja relacionada com a vida do “Todo”.

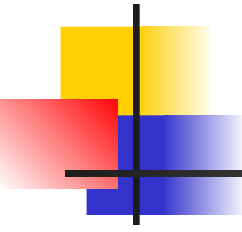


Composição

- Uma forma mais forte de Agregação
- Há uma coincidência da vida das partes
- Uma vez criada a parte, ela irá viver e morrer com ele
- O “Todo” é responsável pelo gerenciamento da criação e destruição das partes

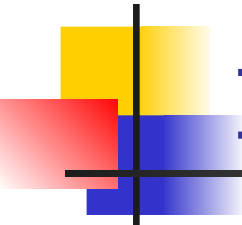
Composição





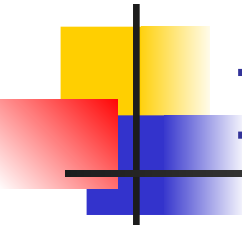
Interfaces

- Uma interface é um conjunto de operações usado pra especificar um serviço de uma classe ou componente
- Diferentemente das classes, as interfaces não especificam nenhuma estrutura
- Interfaces não podem conter atributos

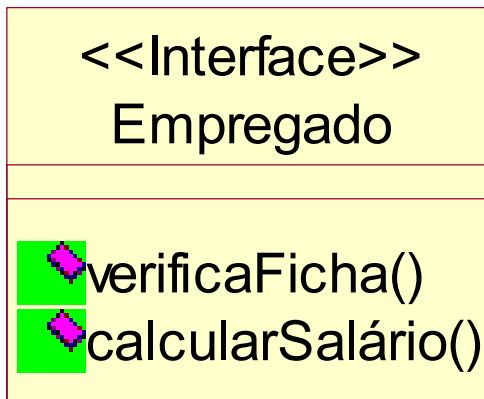


Interfaces

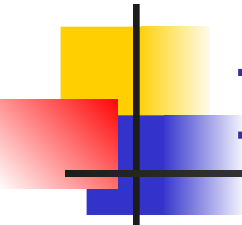
- Com as interfaces, é possível se concentrar apenas nos serviços oferecidos por classe ou componentes
- O uso de interfaces é uma maneira elegante e poderosa de isolar a especificação da implementação
- Uma interface especifica o contrato para uma classe ou componente, sem definir como ele será implementado



Interfaces (Notação)

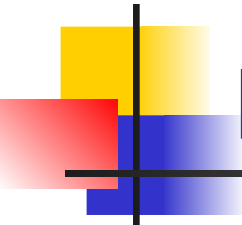


IEmpregado



Interfaces e Realização

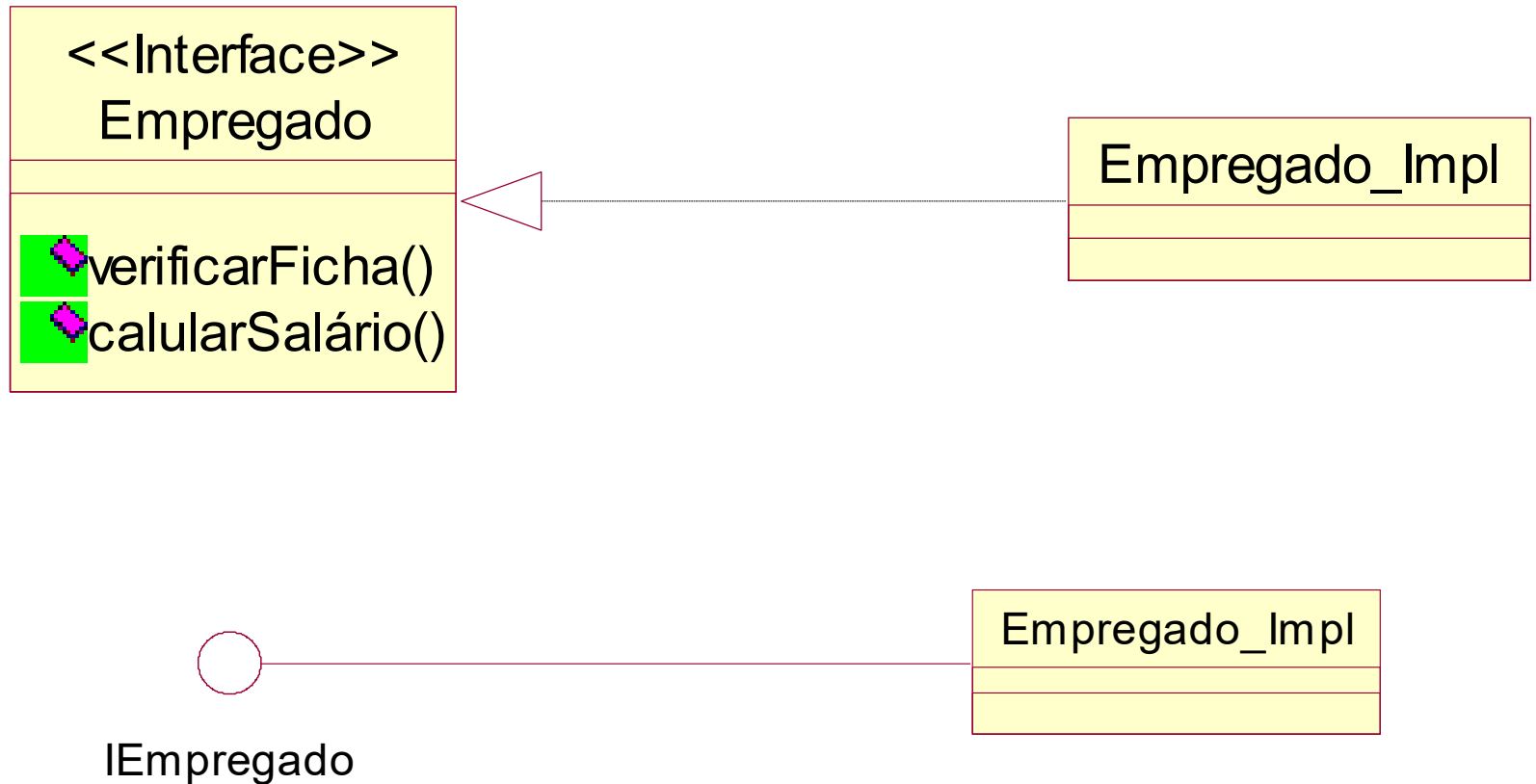
- Realização é uma relação pela qual um elemento especifica o contrato que outro elemento deve implementar
- A realização é um relacionamento entre uma especificação e sua implementação
- É um relacionamento semântico entre classificadores no qual um classificador especifica um contrato que outro classificador garante cumprir
- Está entre dependência e generalização
- Será usado no contexto de interface



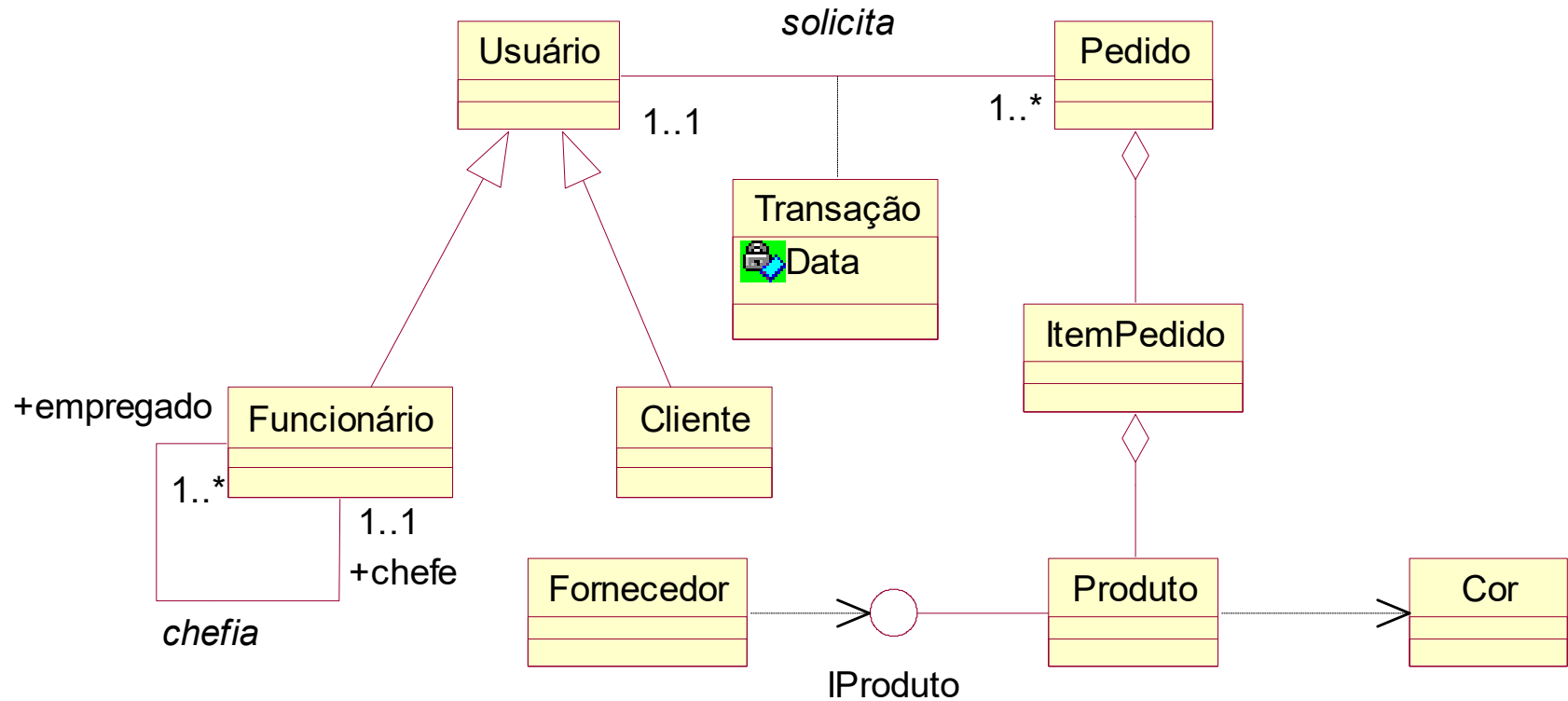
Realização

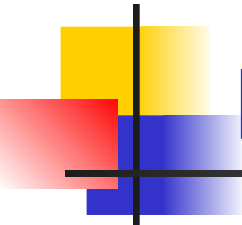


Realização (Notação)



Exemplo





Exercício

- Faça um Diagrama de Classes para representar o sistema da Biblioteca realizado no Diagrama de Casos de Uso.